A Verifiable RLVR Gym for Sequential Planning Under Uncertainty

Imen — *Internal Note, April 2026*

1. The Gap in Current RLVR

Every RLVR gym today tests **static reasoning**: the model sees a complete problem, produces one answer, gets scored. Math, code, logic puzzles — all follow this pattern. No existing gym tests **sequential decision-making under uncertainty**, where the agent must act before the full picture is revealed, each action has a cost, and the optimal strategy depends on future dynamics.

This is a distinct cognitive capability. Consider:

- **Static (math gym):** "The expected return is 0.105 and the fee is 0.15. Should you enter?" → One computation: no.
- **Sequential (our gym):** Same numbers, but the signal persists. Enter now, pay 0.15 once, earn returns for many steps. The answer depends on *how long the signal will persist* — requiring reasoning about dynamics, not just the current snapshot.

The sequential version requires amortisation reasoning: weighing a one-time cost against uncertain future gains. This is the planning capability we target.

2. Why Finance — and Why It's Been Hard

Finance is a natural domain for sequential planning: a portfolio manager deciding whether to enter a trade weighs immediate profit against entry cost, persistence of the opportunity, and exit cost if conditions change. But finance has been resistant to RLVR for a fundamental reason: **you cannot compute the optimal policy when you don't know the true distribution.** Real markets have unknown dynamics, so there is no ground-truth reward to verify against. Existing finance RL environments (e.g., FinRL) use simulation-based evaluation — fine for standard RL, unusable for RLVR.

This is not a limitation of our approach; it is a requirement of RLVR itself. Math gyms use generated equations with known answers. Reasoning Gym uses procedural puzzles. We use a parametric financial model with an exact analytical solution.

3. Our Setup

We build on Bilokon (2026), who formalises a simple but powerful idea: at each time step, an agent observes a noisy signal and decides whether to **activate** (ON) or **deactivate** (OFF) a strategy. Being ON collects the strategy's profit-and-loss (PnL); being OFF earns nothing. Each switch (ON→OFF or OFF→ON) costs a fixed transaction fee λ.

The signal follows an Ornstein-Uhlenbeck (OU) process — it fluctuates around zero with a tendency to revert back. When the signal is positive, being ON is profitable. When it's negative, being ON loses money. The key parameter is κ (mean-reversion speed): low κ means the signal persists (today's value predicts tomorrow's); high κ means it reverts quickly (nearly unpredictable one step ahead).

Why OU? The OU process has Gaussian transitions, which means we can solve the Bellman equation *exactly* via Gauss-Hermite quadrature. No approximation, no Monte Carlo noise — a provably optimal policy for every parameter configuration. This is what gives us the verifiable reward.

Why binary ON/OFF? Bilokon (2026) proves that a single bit — ON or OFF — is the information-theoretically optimal compression of the full market state for predicting strategy performance. It's not a simplification; it's the theoretically justified action space.

4. Where Planning Matters

Without switching costs, the problem is trivial: ON when signal > 0, OFF otherwise. With switching costs, the agent must reason about persistence: "The signal is slightly below my break-even threshold right now, but it's been positive for several steps. Should I pay the cost to switch ON, expecting to earn over multiple future steps?"

This is exactly the planning we want to train.

We ran a full parameter sweep across 1,500 configurations and found:

- **κ is the dominant factor.** It alone explains 46% of the variance in how often the optimal policy disagrees with greedy. At $\kappa = 0.1$ (persistent signal), approximately 13% of decisions require planning. At $\kappa = 0.7$ (fast reversion), only ~1.2% do.
- **The λ/α ratio has a non-monotone effect.** At very low switching costs, both policies agree (switch freely). At very high costs, both agree (never switch). The planning gap peaks at intermediate costs ($\lambda/\alpha \in [0.3, 0.5]$).
- **This gives us parametric control over difficulty.** We define a "planning zone" ($\kappa \in [0.1, 0.25]$, $\lambda/\alpha \in [0.3, 0.5]$) where episodes produce meaningful gradient signal, and a "fast-reversion zone" ($\kappa = 0.7$) as a control condition.

This is analogous to a math gym excluding single-digit addition from the training distribution: episodes where greedy equals optimal produce no training signal for planning.

5. Evaluation Methodology

We introduce a **two-bucket methodology** that separates two capabilities:

- **Easy decisions** (greedy = optimal): Tests comprehension — can the model correctly evaluate the immediate cost-benefit? These are approximately 95% of all decisions.
- **Hard decisions** (greedy $\neq$ optimal): Tests planning — can the model override the locally attractive greedy action when forward-looking reasoning says otherwise?

We further verify hard decisions via **CoT inspection**: does the model's chain-of-thought actually contain forward-looking reasoning (trend recognition, horizon counting, amortisation), or did it match optimal by accident?

6. Results: Claude Opus 4

We evaluated across 4 conditions: 2 parameter zones $\times$ 2 prompt types (original; context prompt adding "you are managing a mean-reversion strategy"), N=30 episodes each, with parameters varied per episode.

	Easy accuracy	Hard decisions	Genuine planning rate
Planning zone ($\kappa = 0.1$–0.25)	97–98%	40	**20%** (8/40)
Fast-reversion zone ($\kappa = 0.7$)	98–99%	23	**4%** (1/23)

Key findings:

1. **Comprehension is solved.** 97–99% easy accuracy across all conditions. The model understands the payoff structure.
2. **Planning exists but is unreliable.** 20% genuine planning rate in the planning zone. On those 8 instances, the model explicitly recognises signal trends, counts remaining steps, computes multi-step amortisation, and overrides the greedy default. It plans like a trader when it plans.
3. **80% greedy default.** On the remaining hard decisions, the model computes the immediate cost-benefit (correctly), concludes "not profitable," and stays with greedy. It does not consider future steps.
4. **The fast-reversion zone validates the methodology.** 4% planning at $\kappa = 0.7$ is the *expected* result — greedy is near-optimal there, so low planning is correct behavior. If we had seen 20% at $\kappa = 0.7$ too, the methodology would be suspect.
5. **Domain-specific prompt has no effect.** The context prompt ("mean-reversion strategy") produces statistically identical results. The bottleneck is not financial knowledge — the model already knows what mean-reversion is. The bottleneck is constructing the forward-looking reasoning scaffold from sequential observations.

7. Why This Matters for RLVR Training

The 20% genuine planning rate is exactly the right signal for GRPO:

- **Not 0%** — correct reasoning paths exist in the model's distribution, so GRPO has something to reinforce. No cold-start SFT needed.
- **Not 100%** — there is a clear capability gap to close. The model has the knowledge (comprehension is solved) and the latent ability (20% planning), but doesn't reliably activate it.
- **The 80% failure mode is consistent** — always greedy reasoning, never hallucinated planning. This means GRPO would push against a specific, identifiable behavior pattern.

The gym provides: exact verification (no reward noise), parametric difficulty control (episodes that actually test planning), and a diagnostic methodology (two-bucket + CoT) to measure progress.

Integration is minimal: one function `reward_fn(completions, problems) → list[float]`, compatible with TRL/veRL GRPO pipelines.

8. Beyond Finance

The planning capability tested here — amortising a one-time cost against uncertain future gains under a noisy, persistent signal — is not specific to finance. The same structure appears in:

- **Energy dispatch:** activating/deactivating generators with startup costs under fluctuating demand
- **Supply chain:** onboarding/offboarding suppliers with switching costs under variable demand signals
- **Treatment protocols:** starting/stopping treatments with side-effect costs under evolving patient indicators

The domain-specific prompt result supports this: the bottleneck is sequential reasoning, not domain knowledge. Training on the financial gym could develop planning capabilities that transfer to any domain with the same cost-amortisation structure. The framework extends to other signal processes and PnL functions by changing only the transition kernel and payoff mapping.

9. Current Status

- ✓ Gym implemented, tested, open-source
- ✓ Bellman solver verified (T=1 reduces to greedy, threshold matches analytical prediction)
- ✓ 1,500-config parameter sweep completed
- ✓ 4-condition LLM evaluation with raw text archived
- ✓ Single-function TRL integration
- ☐ RLVR training not yet conducted
- ☐ Transfer to other domains not yet tested

The gym is ready for training experiments. The question is whether RLVR can push the 20% planning rate higher — and whether the capability transfers.